DATA STRUCTURES AND ALGORITHMS · PYTHON · 2026/2027 FIRST SEMESTER · LAB 1

Sheet 1 - lists, loops and your first search

Solve it yourself, after lab 1. Hand it in at the start of lab 2.

Where your work goes. One PyCharm project, `Desktop\DSA`. Inside it, the folder `sheet01`. One file per question, with the name given next to each question (`q01_hello.py` ...).

By-hand answers go in the file too, as `#` comments at the top, written **before** you run anything. There is nothing to print or write on paper.

Required: Q1-Q9. **Optional stretch:** Q10-Q11. **Do not use** `max()`, `.index()`, `.count()`, `sorted()`, or a membership test such as `target in values` - the point is to write the loop yourself. (The `in` of `for i in range(...)` is of course allowed.)

Hand in: zip the whole `sheet01` folder as `sheet01_<your ID>.zip` - the `.py` files, not screenshots - and send it the way your TA announces in lab 1.

For Q5, Q7 and Q8, write these five lines as comments above the function, before its code. Use them on any other question where you cannot see how to start.

HAVE	the inputs, and what you may assume about them (sorted? empty? repeats?)
WANT	the exact result - including what happens when there is nothing to find
TRY	one small example, solved by hand, before any code
STEPS	what your hands just did: what you remember, what you repeat, when you stop
CHECK	the test inputs that could catch a mistake, with the answers you expect

Q1 Hello, and find it again `CODE` — `q01_hello.py`

Make `sheet01/q01_hello.py` so that it prints exactly:

```
Hello from DSA - sheet 1
```

Run it. Close PyCharm, open it again, open the `DSA` project, find the file and run it again.

Q2 Indexes and len `BY HAND` `CODE` — `q02_indexes.py`

```
seats = [4127, 3051, 5310, 2268, 4890]
```

First, as comments, write what you expect each of these to be: `len(seats)`, `seats[0]`, `seats[2]`, `seats[len(seats) - 1]`. Then add a `print` for each and run. Next to any prediction that was wrong, add a comment saying why.

Q3 Trace a linear search BY HAND

q03_trace.py

```
def find_index(seats, target):
    for i in range(len(seats)):
        if seats[i] == target:
            return i
    return -1

seats = [4127, 3051, 5310, 2268, 4890]
```

For `find_index(seats, 2268)` and then `find_index(seats, 9999)`, write the trace as comments, one row per comparison:

i	seats[i]	equal to the target?	comparisons so far
0	4127	no	1
...			

End each trace with what the function returns, and how many comparisons it made in total.

Q4 Put the lines in order BY HAND CODE

q04_order.py

These are the lines of `find_index`, scrambled. The indentation is correct. Type them into the file in the right order, then run it.

```
    return -1
print(find_index([4127, 3051, 5310], 9999))
        if seats[i] == target:
def find_index(seats, target):
            return i
print(find_index([4127, 3051, 5310], 5310))
    for i in range(len(seats)):
```

It should print `2` and then `-1`. In a comment: **why do the two `return` lines sit at different depths?**

Q5 find_index CODE

q05_find_index.py

Write the five plan lines as comments, then the function. It returns the index of the **first** position where `target` appears in `seats`, or `-1` if it is not there. `-1` is this function's signal for "not found".

```
def find_index(seats, target):
    # your code here


seats = [4127, 3051, 5310, 2268, 4890]
print(find_index(seats, 5310))
print(find_index(seats, 9999))
```

It must print `2`, then `-1`.

Q6 Tests that catch mistakes CODE

q06_tests.py

Copy your `find_index` into this file and print its result for each case below, next to the expected answer. Then, for each case, add a comment naming a mistake that this test would catch.

case	call	expected
first	`find_index(seats, 4127)`	`0`
middle	`find_index(seats, 5310)`	`2`
last	`find_index(seats, 4890)`	`4`
absent	`find_index(seats, 9999)`	`-1`
empty list	`find_index([], 4127)`	`-1`
repeated	`find_index([3051, 4127, 3051], 3051)`	`0`

`seats` is the list from Q5.

Q7 Count the matches CODE

q07_count.py

Write `count_occurrences(values, target)`. It must **return** the number of times `target` appears in `values` - `0` when there are none, including for an empty list. Plan it first. Unlike `find_index`, it must not stop at the first match. Print each result, like `print(count_occurrences(grades, 7))`.

call, with `grades = [7, 10, 7, 9, 7, 4]`	expected
`count_occurrences(grades, 7)`	`3`
`count_occurrences(grades, 10)`	`1`
`count_occurrences(grades, 5)`	`0`
`count_occurrences([], 7)`	`0`

Q8 Count the comparisons CODE

q08_comparisons.py

Copy `find_index` and rename it `find_index_counting`. Add a counter that goes up by one every time the `if` compares a value with the target, and print `comparisons:` and the count just before each `return`. With the list from Q5:

call	prints	returns
`find_index_counting(seats, 4127)`	`comparisons: 1`	`0`
`find_index_counting(seats, 2268)`	`comparisons: 4`	`3`
`find_index_counting(seats, 9999)`	`comparisons: 5`	`-1`

Then make three longer lists and search each one for `-1`, which none of them contains. `list(range(10))` is the list `[0, 1, 2, ..., 9]` - ten numbers:

```
find_index_counting(list(range(10)), -1)
find_index_counting(list(range(100)), -1)
find_index_counting(list(range(1000)), -1)
```

Fill in a table of list length and comparisons as comments. In one sentence: what happens to the comparisons when the list is ten times longer?

Q9 Repair an AI draft BY HAND CODE

q09_ai_draft.py

Consider this hypothetical AI-generated draft of `find_index`. It runs without errors.

```
def find_index_draft(seats, target):
    for i in range(len(seats)):
        if seats[i] == target:
            return i
        else:
            return -1
```

a. Give a list and a target for which it returns the **right** answer.

b. Give a list and a target for which it returns a **wrong** answer, and the answer it should have given.

c. In one comment line: what exactly is wrong?

d. Write the corrected function and run it on your input from (b).

e. Run the draft on an empty list. What comes back, and why is that neither an index nor -1? *(Hint: the guide, chapter 6, "What could fool you".)*

Q10 The largest value OPTIONAL CODE

q10_largest.py

Write `largest(values)` for a list with at least one value, without `max()`. **New idea:** start from a real value in the list, `values[0]`, not from 0 - the third test shows why.

call	expected
`largest([4127, 3051, 5310, 2268, 4890])`	`5310`
`largest([7])`	`7`
`largest([-8, -3, -12])`	`-3`

Q11 Is it sorted? OPTIONAL CODE

q11_sorted.py

Write `is_sorted(values)`: `True` if every value is less than or equal to the one after it, otherwise `False`. **New idea:** compare neighbours, `values[i]` and `values[i + 1]` - so where must the loop stop?

call	expected
`is_sorted([2268, 3051, 4127, 4890, 5310])`	`True`
`is_sorted([4127, 3051, 5310])`	`False`
`is_sorted([7, 7, 9])`	`True`
`is_sorted([])`	`True`
`is_sorted([5])`	`True`

Stuck? The lab 1 guide, chapter 8, says how to start each question. Every expected result on this sheet was produced by running a solution on Python 3.14, the version on the lab PCs.